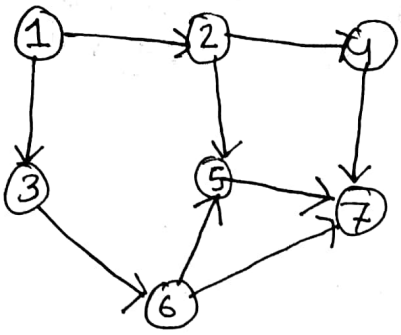
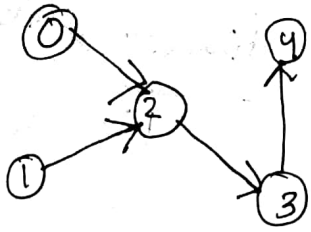
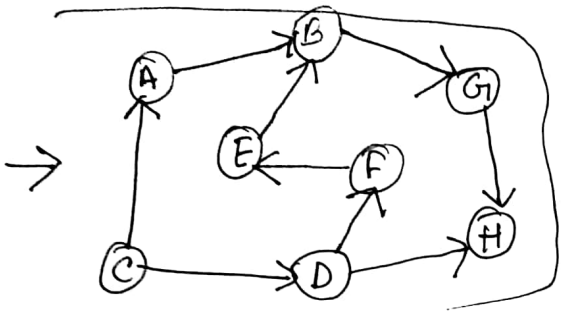
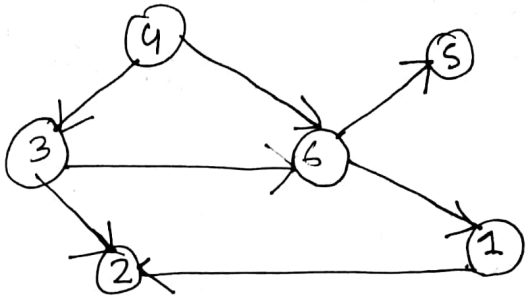
2(a)



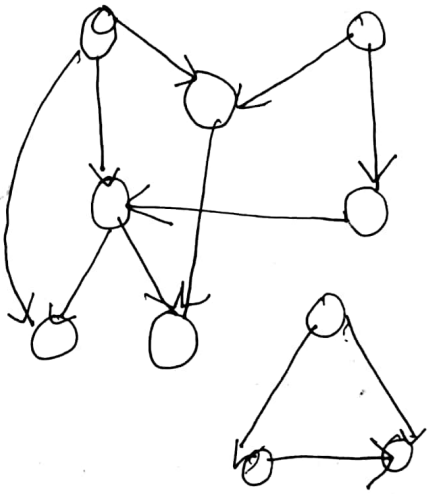






2(b)

	A	B	C	D	E	F	G	H
A	0	1	0	0	0	0	0	0
B	0	0	0	0	0	0	1	0
C	1	0	0	1	0	0	0	0
D	0	0	0	0	0	1	0	1
E	0	1	0	0	0	0	0	0
F	0	0	0	0	1	0	0	0
G	0	0	0	0	0	0	0	1
H	0	0	0	0	0	0	0	0



A → B → NULL

B → G → NULL

C → A → D → NULL

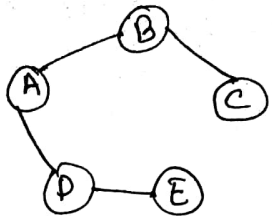
D → F → H → NULL

E → B → NULL

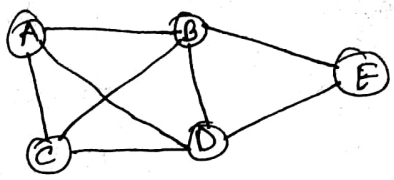
F → E → NULL

G → H → NULL

2c

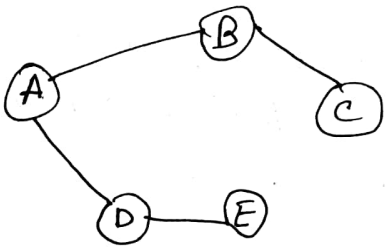


sparse



Dense

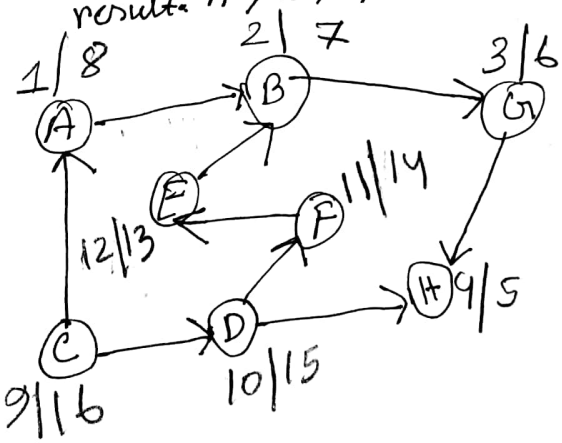
2(d) DFS



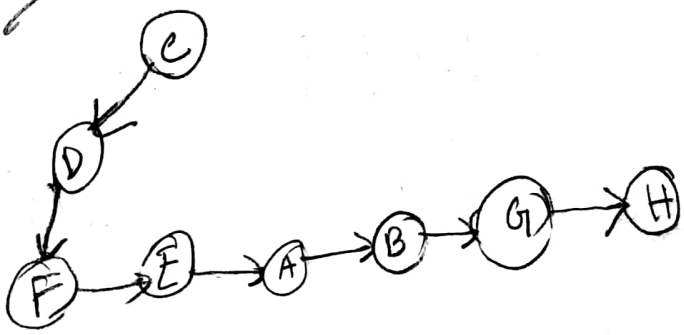
stacks

result: A, B, C, D, E

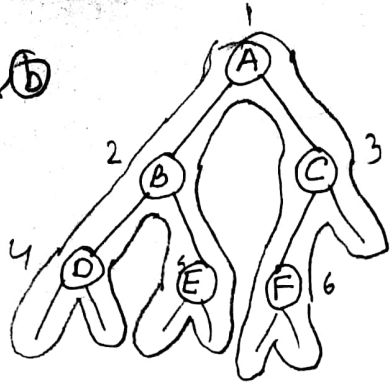
2e



Topological sorting



3(a) (b)



2 InOrder: D, B, E, A, F, C

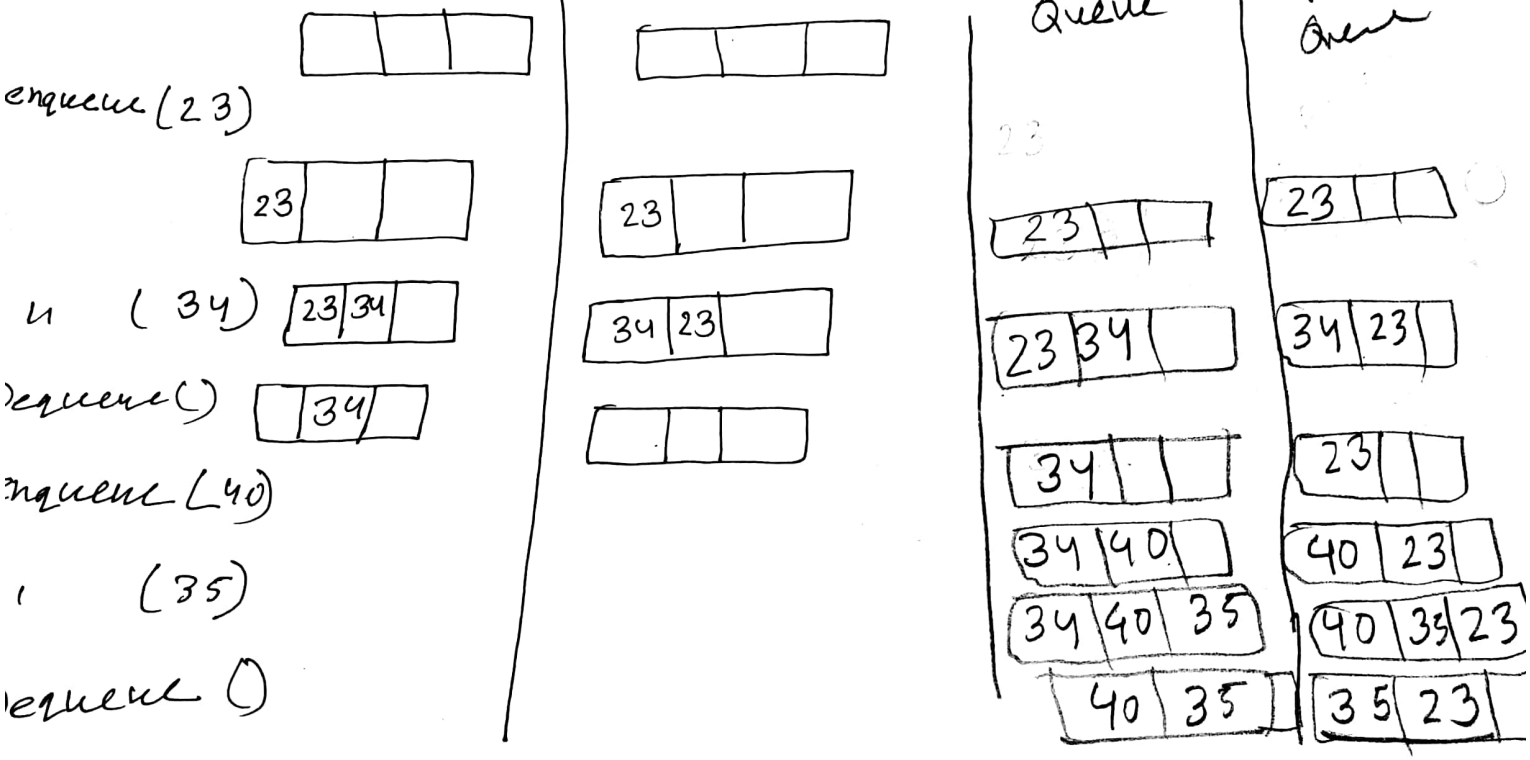
1 PreOrder: A, B, D, E, C, F

3 PostOrder: D, E, B, F, C, A

LevelOrder: A, B, C, D, E, F

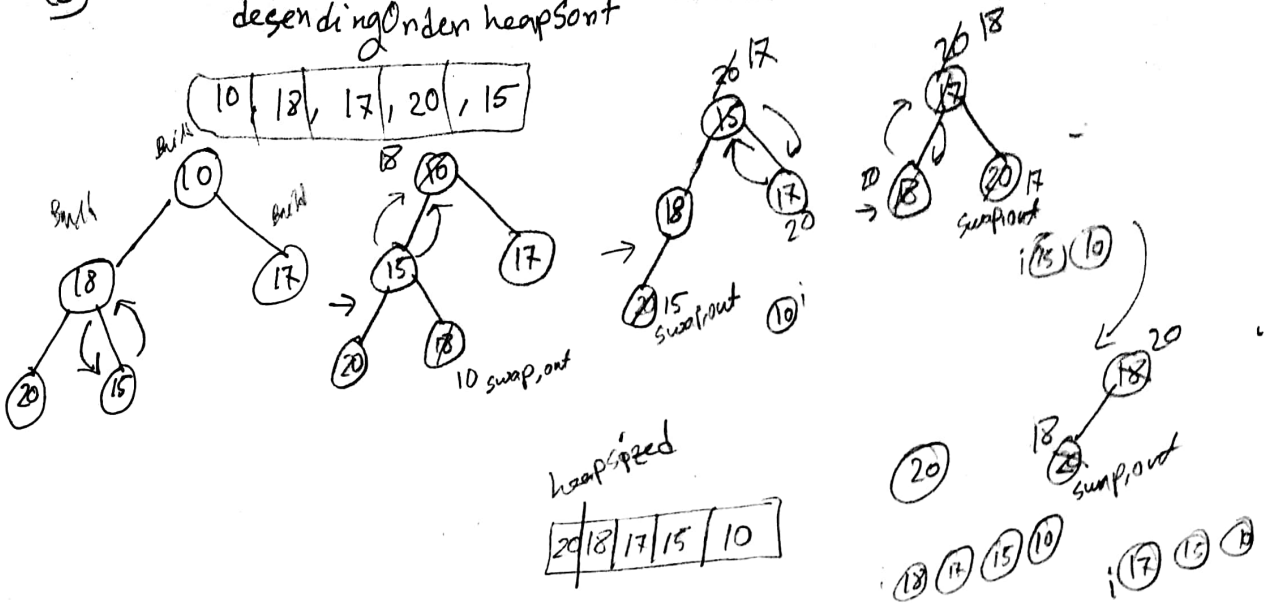
height: Number of edges in longest path from root to a leaf node, height of the ~~three~~ is 2.

3 (c) array size, m = 3, → queue → Priority Queue



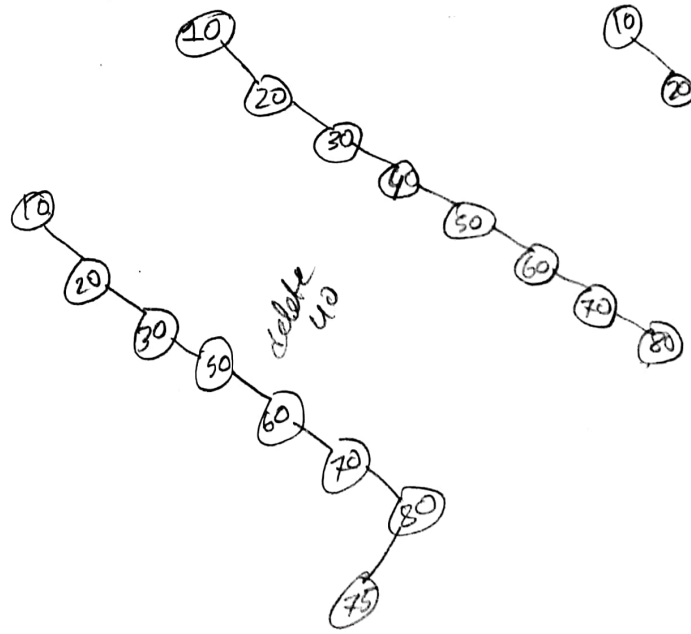
3 (d) Decending Order

descendingOrder heapSort

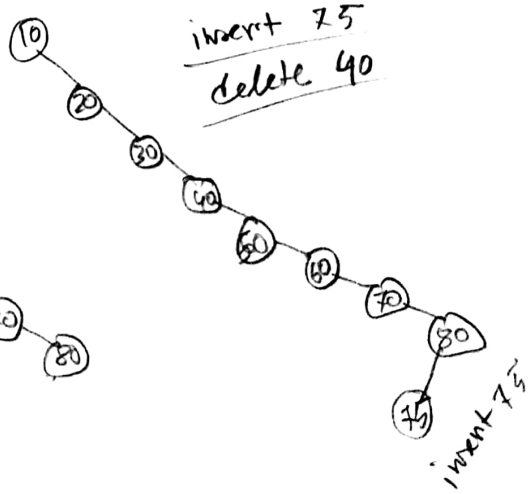


4 (a) Construct a binary search tree

10, 20, 30, 40, 50, 60, 70, 80



(b)



final - Spring 23

Tower of hanoi

s, d, a
n-1, s, a, d
n-1, a, d, s

1 (c) Algorithm

```
Void Hanoi(int n, int s, int a, int b){
if(n==1){
        printf("%d→%d", s, d);
}
else{
Hanoi(int n-1, int s, int d, int a);
Printf("%d→%d", s, d);
Hanoi(int n-1, int a, int s, int d);
}
}
```